

DEG: A Disturbance Event Generator for GCBM

Paradis, Gregory
gregory.paradis@ubc.ca

August 14, 2018

Abstract

This document describes a *disturbance event generator* (DEG) software framework and prototype implementation, which generates spatially-explicit disturbance input data for the GCBM carbon budget simulation model.

1 Introduction

The Canadian Forest Service (CFS) is currently developing the next-generation carbon budget model (i.e., the *General Carbon Budget Model*, or GCBM). The GCBM modelling platform will eventually be the successor to the *Operational-Scale Carbon Budget Model of the Canadian Forest Sector* (CBM-CFS3), which was released in 2005.

One of the key inputs to GCBM (and other similar carbon budget models) is a projected schedule of *disturbance events* (i.e., the type, timing, and location of events, defined spatially in terms of a rasterized grid of even-sized pixels). Events can include both anthropogenic disturbances (e.g., deforestation, afforestation, reforestation, harvesting, thinning, slashburning, fertilization, prescribed fire, or the introduction of invasive pests) and natural disturbances (e.g., fire, mortality from native and invasive pests, blowdown, flooding, climate change, etc.). Indeed, when multiple future CBM scenarios are considered on a given forest landbase, the disturbance schedule is one of the key inputs that can be varied to model different policy options or mitigation strategies.

CBM-CFS3 includes a built-in software module for scheduling disturbance events, which uses a priority-rule-based heuristic simulation method to generate disturbance schedules. GCBM does not include similar functions for generating disturbance schedules, so it must rely on external tools schedule disturbance events. This document describes a *disturbance event generator* (DEG) software framework and prototype implementation developed to fill this gap.

The DEG framework is a collection of software components, which include specialized functions to automate the task of importing, pre-processing, and compiling input data required to model multiple *management units* (MU), building models from this data, running these models (including diagnostic reports), and saving output to raster data files that can be used as input for GCBM.

The prototype implementation of the DEG framework described here is custom-built to generate GCBM input disturbance events for the province of British Columbia (BC), in the context of the PICS Forest Carbon Management Project¹—naturally, many of the details and model parameters documented here are specific to BC or the PICS project. However, the underlying software framework is quite flexible, and was designed to be readily adaptable for use at different scales and geographic locations.

Note that DEG output feeds directly into GCBM input, and that GCBM also simulates the evolution of the forest system over time. The process used to simulate the evolution of forest state over time in the DEG has deliberately been designed to be congruent with the simulation process in GCBM. Thus, if the analyst inputs equivalent data to the DEG and GCBM, it is possible to produce coherent simulations from both models. However, the DEG and GCBM are distinct software systems that are designed to be run sequentially—the only direct link between the two systems is that the output from a DEG simulation (i.e., the GeoTIFF files containing disturbance event data) is used as input to GCBM. There is little in the way of automated error-checking, so coherence must be achieved through careful coordination of input files and model parameters in both DEG and GCBM—the analyst running the DEG is solely responsible for coherence (or lack thereof) between DEG and downstream GCBM simulations.

The remainder of this document is organized as follows. The DEG modelling framework is presented in §2, followed by a description of the DEG software implementation details in §3. Supplementary information is provided in an Appendix.

2 Framework

This section describes the DEG framework. We first provide a high-level overview of the purpose and nature of the DEG in §2.1, followed by more detailed description of each step of the modelling process in §2.2, and modelling details for specific disturbances (i.e., wildfire, harvesting, and slashburning) in §2.3.

2.1 Overview

The DEG software framework automates the process of generating *disturbance event* data, for input into the GCBM simulation model, for multiple scenarios, MUs, disturbances, and time steps. The DEG software prototype runs on a server, and is accessed with a web browser, using a Jupyter Notebook interface to interact with the model. The software prototype simulates wildfire, harvesting, and slashburning disturbances in 1-year timesteps at 1-hectare pixel resolution, for all 38 TSAs in the public forest of BC. Although the prototype is purpose-built to generate GCBM disturbance input data (specifically, for simulations run in the context of the PICS project), the more general framework on which

¹For more information, see <https://pics.uvic.ca/>.

it is built can be readily adapted for use in other modelling projects that require spatially-explicit projections of disturbance over time.

For a given scenario, the DEG outputs a series of GeoTIFF² georeferenced raster images files—one file for each combination of MU, time step, and disturbance type. These GeoTIFF files are imported into GCBM, and used to simulate carbon budgets of future forest management scenarios. The sole purpose of the DEG is to generate this pixelated disturbance event data for GCBM.

Figure 1 provides a schematic overview of the DEG modelling process.

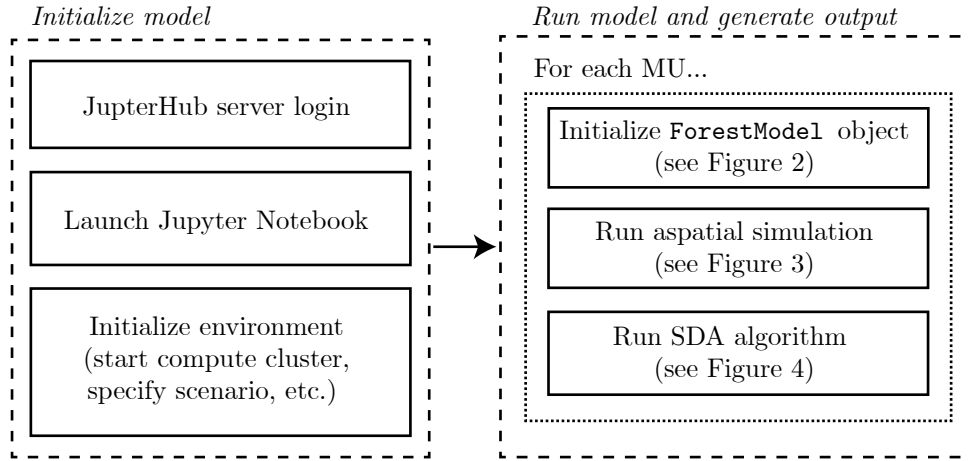


Figure 1: Schematic representation of DEG modelling workflow.

Fire and harvesting disturbances are modelled in the aspatial modelling step using a *two-stage rolling-horizon replanning simulation* (RHRS) approach. This allows us to model wildfire disturbance events differently than harvesting disturbance events, taking into account the random nature of wildfires and the deliberate nature of harvesting.

Forest managers have little control over the timing, location, size or intensity of future wildfire disturbance events, but we do have considerable data describing past events. For each combination of MU and planning period, we specify a wildfire area target, which is compiled from historical burn data in each MU. At each iteration of the simulation, the DEG selects random stands to burn (see §2.3.1 for a more detailed description of wildfire disturbance modelling), such that the sum of the area of these stands is similar to the target area specified for that period.

After wildfire has been simulated for the current period, the DEG schedules

²For more information on the GeoTIFF file format, see <https://trac.osgeo.org/geotiff/>.

harvesting by solving a multi-period optimization problem, which either *maximizes harvest volume* or *minimizes harvest area*, subject to various constraints (see §2.3.2 for a more detailed description of harvest disturbance modelling, and also Appendix A1 for a mathematical formulation of the optimization problem that is solved repeatedly by the DEG to yield the final harvest disturbance schedule). Input data for the harvest scheduling model includes MU-wise periodic harvest volume targets, and parameters specifying allowable deviation from these targets and allowable periodic variation in area harvested (i.e., a form of *even flow* constraint). Thus, wildfires occur in the first instant of the current planning period, and harvesting events occur in the instant after that. Because harvesting events are scheduled *after* wildfire events in the simulation model algorithm, the harvest scheduling model is aware of current-period fires and can schedule around them—the harvest scheduling model is not aware of fires that will occur in future periods, because *future wildfire events have not been scheduled yet*.

GCBM expects disturbance event pixel input data (i.e., the GeoTIFF output files from the DEG) to be organized by MU, year, and disturbance type. To accommodate this, the model must track disturbances using the same aggregation scheme (i.e., by MU, year, and disturbance type). The current DEG prototype decomposes the provincial-scale problem into MU-wise subproblems, and solves each subproblem separately. This MU-wise decomposition makes it relatively easy to implement parallel processing in the DEG, which can provide a substantial reduction in wall-clock processing times if the DEG is run on a powerful multi-core server. A disadvantage of this MU-wise decomposition is that we cannot model any interaction between MUs³.

2.2 Detailed description of DEG modelling process

Figures 2 through 4 show more detailed views of the `ForestModel` object initialization, aspatial disturbance modelling, and SDA steps of the process shown in Figure 1.

First, the analyst initializes the modelling environment. This involves logging into the server, launching an appropriate notebook (an analyst may have created more than one DEG notebook for various reasons), and initializing the modelling environment within the notebook. This last step can include launching an *IPython parallel computing cluster* (if the parallel processing functions in the DEG are going to be used), specifying a scenario to model, and possibly modifying parameter values in the notebook. The DEG notebook interface is

³The DEG software framework can also handle modelling many MUs together in a single model, which has the advantage of allowing some cross-MU communication. For example, if we model all the MUs in a province in a single monolithic model, we could specify periodic harvest volume targets at the provincial or regional levels (instead of by MU). Although the multi-MU modelling functions are fully implemented and functional, the software implementation of these functions in the `ws3` library requires large amounts of memory (RAM) and relatively long processing times. We hope to improve the efficiency of the `ws3` software implementation in the future, at which point these multi-MU modelling functions can be used to model a range of new provincial-scale scenarios.

very flexible, and allows the user to easily customize the DEG interface and functionality.

Once the modelling environment is initialized, the analyst runs the model and generates output. As discussed earlier, the DEG processes each MU as an independent subproblem. Subproblems can be processed sequentially, or in parallel—the end result is the same. Sequential processing makes it easier for the user to monitor processing of each MU, which can be useful when debugging problems or testing new parameter values. Parallel processing is faster (up to 16 times faster on the current server), but makes it more difficult for the user to monitor and debug problems with individual MUs. For each MU, the DEG initializes a **ForestModel** objects (see Figure 2), runs the aspatial simulation algorithm (see Figure 3 and §2.2.1), and finally runs the SDA algorithm (see Figure 4 and §2.2.2).

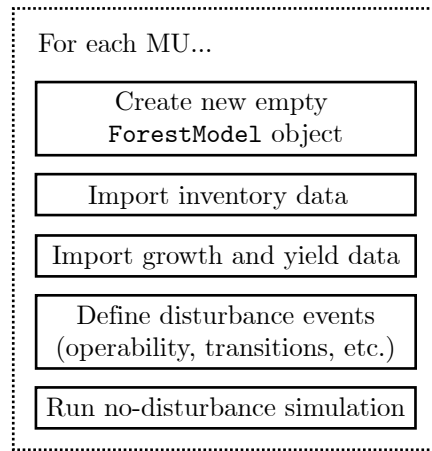


Figure 2: Detailed view of the model initialization step.

The **ForestModel** class is the core class of the **ws3** library. For each MU, the DEG creates a new **ForestModel** object, which is used to model the aspatial (i.e., area-based) disturbances in the next step. Each **ForestModel** object needs to be loaded with an initial inventory, yield curves, and disturbance event definitions (i.e., operability, transitions, etc.).

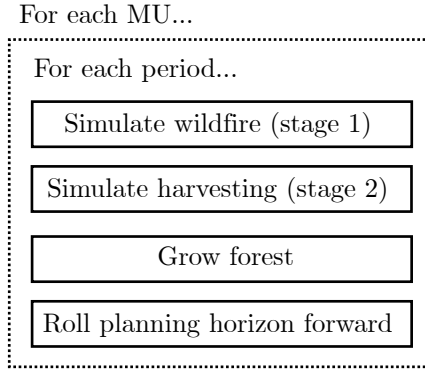


Figure 3: Detailed view of the aspatial disturbance modelling step.

2.2.1 Aspatial disturbance area simulation process details

The aspatial disturbance modelling step is modelled using a two-stage rolling-horizon replanning simulation (RHRS) algorithm. Stage 1 of the RHRS algorithm models wildfire. Stage 2 of the RHRS algorithm models harvesting. For each MU, the `ForestModel` object created in the previous step is manipulated within the RHRS algorithm to produce the aspatial *disturbance schedule*. Output from the aspatial forest model is expressed in terms of area disturbed by disturbance type, development type, and age class, in a given planning period. The aspatial disturbance schedule does not specify which pixels are disturbed—pixel-wise allocation is handled in a subsequent (SDA) processing step.

A *development type*, or stand group, is considered to be homogenous for the purposes of simulating growth, eligibility for disturbance events (including both anthropogenic and natural disturbance), and post-disturbance transitions. For each development type, the aspatial model requires input data describing initial inventory of every stand, growth and yield curves for each stand type, age-indexed eligibility of stands to different disturbance types, and post-disturbance transitions to different development types. When correctly compiled into a model, this data allows us to simulate the initial state of a forest, its evolution over time, and its response to various disturbances.

In stage 1 of the RHRS algorithm, the DEG simulates wildfire disturbance events. When stage 1 is completed, the model locks down fire events and proceeds to stage 2.

For each MU...

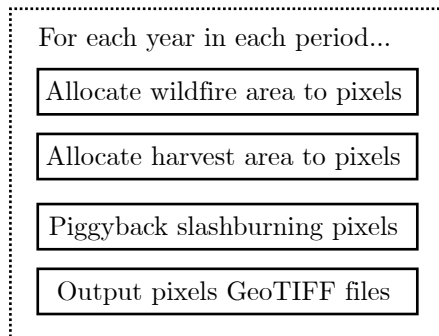


Figure 4: *Stage 2*: Detailed view of the SDA step (repeated for each management unit (MU)).

In stage 2 of the RHRS algorithm, the model schedules harvesting for the next $\max(1 + L, N - i)$ periods⁴ using an optimization-based harvest scheduling model⁵.

After stages 1 and 2 are complete, the model grows the forest (for one period) and rolls the planning horizon forward (for one period). This is repeated for as many iterations as there are periods in the planning horizon.

2.2.2 Spatial disturbance allocation (SDA) process details

Output from the aspatial disturbance area simulation process is an *aspatial schedule* of harvesting and wildfire disturbances—this takes the form of a list of aspatial disturbance events (i.e., a *schedule* of events). Each item in the disturbance schedule specifies the *development type* key (an n -length tuple of string values, one value per *theme*), the ageclass, the action code, the area disturbed, and the 10-year planning period in which to simulate this disturbance.

In the *spatial disturbance allocation* (SDA) step, the DEG disaggregates the aspatial disturbance schedule into 1-year time steps and 1-hectare pixels. The SDA step also includes simulation of piggyback slashburning disturbances (see §2.3.3 for more details). Note that DEG output pixel size and time step length parameters must be set to match parameters used in GCBM downstream, or DEG output will not be coherent with GCBM model configuration.

The SDA algorithm processes each line in the aspatial disturbance schedule sequentially (sorted by period). For a given line in the aspatial disturbance schedule, the SDA algorithm must first divide the target area by the period

⁴Where L is the maximum number of look-ahead periods, i is the current iteration index and N is the number of periods in the global simulation horizon. In other words, the RHRS algorithm schedules harvesting for a user-defined number of look-ahead periods (current prototype uses N), but truncates this look-ahead horizon to avoid looking past the end of the global planning horizon. This is important to mitigate undesirable modelling artifacts.

⁵The mathematical formulation of the optimization model used to schedule harvest disturbances is documented in Appendix A1.

length (to disaggregate from periods to years), and round this area to the nearest multiple of pixel area then divide by the area of one pixel (to convert area to pixel count). The algorithm then attempts to disturb the target pixel count in each year of the current planning period, proceeding sequentially through a list of years—by default time steps in a period are processed ordinally (i.e., first to last), but may be processed in randomized order in the case of sub-pixel target area (which sometimes occurs when wildfire target area is low) to avoid inducing a sawtooth pattern in disturbed area (with the point of each tooth at the first year of each period).

To disturb pixels, the SDA algorithm must read from and write to a cube of data, which represents multiple attribute layers of an $m \times n$ matrix of forest pixels, in a given period. The attribute layers store development type⁶, age class, and block identifier⁷. Before running the SDA algorithm, the data cube is initialized to first-period inventory (pre-disturbance). At each iteration, the SDA algorithm queries the data cube for all pixels matching the targeted development type and age class, and builds a list of block ID codes found in these pixels. Then, a random block ID code is selected from this list, and all pixels in that block are selected and disturbed. The algorithm continues to disturb entire blocks as long as the target disturbance area (for the current 1-year time step) has not been met. If the area of a selected block exceeds the outstanding disturbance area target, the algorithm discards the block (it may still be used later in a subsequent 1-year time step) and selects a new (hopefully smaller) block. If only one block remains in the list and its area exceeds the outstanding disturbance area target, a random subset of the pixels in the block are disturbed such that the target is exactly achieved.

As pixels are disturbed (at each 1-year time step in a given period), pixel attributes are updated to match the post-action transition rules defined in the aspatial forest model (e.g., a *clearcut and plant* disturbance might reset the age attribute of a pixel to 0, and update the development type hash code to match the development type of the new plantation). When all lines in the aspatial disturbance schedule have been processed for a given period, the SDA algorithm *grows* the forest by incrementing the value of all pixels in the age layer by 1.

Because the SDA algorithm uses all the same logic rules as the aspatial model to transition and grow pixels after disturbance, the aspatial solutions generally

⁶We use a numerical matrix library (`numpy`) to implement the data cube, which allows for relatively fast query and update operations. Some attributes, such as development type, are expressed as a tuple of string values which must be encoded to a numerical value before being stored in the data cube. We convert development type key tuples to integer values by taking the first 4 bytes of the *md5* hash value of the development type key tuple. This truncation is necessary to ensure compatibility with the `rasterio` library we use for GeoTIFF raster file read and write operations (the largest data type we can write to a GeoTIFF layer using the `rasterio` library is a 32-bit unsigned integer).

⁷The SDA algorithm can be constrained to disturb entire *blocks* (i.e., groups of pixels), which offers some measure of control over the output patch size distribution of the forest landscape simulated by the DEG if blocks are constrained to be connected subgraphs of the pixel matrix. Our prototype BC DEG implementation uses forest inventory polygons as blocks—the software framework will work with any other pixel blocking scheme.

remain feasible, even after several periods of sequential updates. However, side-effects of the SDA algorithm may induce infeasibility in later periods (e.g., from rounding error induced by discretization of linear variables to whole pixels and other causes). Although the DEG software implementation does include some heuristic functions to handle minor infeasibility problems, the analyst running the DEG is ultimately responsible for balancing aspatial optimization model formulation with SDA algorithm stability issues—this is a complex task, and discussion of the details of achieving this balance is outside the scope of this document.

The DEG software implementation also includes diagnostic functions that output summary performance reports that the analyst can use to help determine if the quality of model output is acceptable (or help diagnose the root cause of instability if the output is not acceptable). The notebook-based DEG interface makes it relatively easy for an expert analyst to extend or modify DEG diagnostic functions as needed.

2.3 Disturbance modelling details

This section provides more details on the implementation of the different kinds of disturbance types in the DEG prototype developed for the PICS project.

2.3.1 Wildfire disturbance modelling details

Wildfire disturbances are first modelling in the RHRS algorithm in the first (aspatial) step of DEG processing. The algorithm burns an equal proportion of each age class in the current inventory—the proportion is equal to the quotient of periodic burn area target (provided as an input parameter) and total MU area (deduced from stand geometry input data). This equal-proportion-by-age-class wildfire simulation methodology is adapted from a method described in Savage et al. (2010). Although this equal-proportion-by-age-class wildfire simulation methodology is relatively robust and simple to implement, it does induce fragmentation of disturbance target area across all combinations of development type and age class. In some cases, this induces disturbance area events that are smaller than the area of one pixel in the model, which complicates the SDA step⁸. Note that other non-anthropogenic disturbances (e.g., climate change, forest pest epidemics, blowdown, etc.) could also be modelled in the first stage of the RHRS algorithm in the aspatial disturbance scheduling step of the DEG processing workflow.

⁸For example, if the coincidence of MU area, number of development types, number of non-null-area age classes, and wildfire area target tends to yield 0.5 hectare events in a 1-hectare-pixel model, then systematically rounding down will tend to yield substantially less wildfire-disturbed area than targeted. Conversely, rounding up would tend to yield substantially more wildfire-disturbed area than targeted. The DEG includes heuristic functions to detect and deal with this special case. The heuristic function parameters have been tuned to work well with the BC sample data we used to develop the prototype, however performance may be poor with other datasets. Analysts must therefore be vigilant for DEG output problems, with special attention to MUs that tend to generate sub-pixel wildfire events, and be prepared to re-tune sub-pixel disturbance heuristic function parameters as required.

Wildfire disturbances are only modelled for the first period of the rolling horizon at each RHRS iteration. At the end of the first stage, period-1 wildfire disturbances are *locked into* stage-2 initial inventory used to schedule harvest disturbances—so, the harvest scheduling algorithm must schedule around period-1 fires.

2.3.2 Harvesting disturbance modelling details

As explained in §2.1 above, harvest disturbances are scheduled using a Model I optimization problem formulation. See Appendix A1 for a detailed description of the mathematical formulation of the optimization problem. In general terms, the harvest scheduling optimization problem finds the optimal sequences of silviculture prescriptions for each development type. The optimization model built into the DEG is quite flexible, and can accommodate a range constraints and objectives.

The optimization problems used to schedule harvest disturbances use a linear programming (LP) formulation, and can generally be solved very quickly (on the order of 100 ms) by an external solver. So, the processing bottleneck in the optimization problem subroutine is not related to solving the problems, but rather in the upstream problem building step. Depending on problem size and DEG parameter configuration, building a multi-period harvest schedule optimization problem for one MU can require substantial CPU processing time and memory (RAM).

Memory requirement for the problem-building function tends to increase geometrically with the number of planning periods. This is largely due to the dendritic tree (i.e., connected acyclic directed graph) data structures used to store information on all possible future states of each development type. `ws3` builds one state tree for each combination of development type and age class in the initial forest inventory. Nodes represent system states at specific moments in time. The time gap between nodes is equal to the period length. The root node represents initial state (i.e., at the start of period 1). Each node has at most one inbound directed arc, connecting it to a parent node, and may have any number of outbound arcs linking it to child nodes (in the next period). Each arc represents an eligible action (including the null action, which simply grows the stand) that can be applied to the parent node to spawn a child node. Each leaf node has a single *path* linking it back to the root node. Each path corresponds to one decision variable x_{ij} in the Model I optimization problem formulation (see Appendix A1). The depth of the tree (i.e., the number of nodes in each path) is equal to the planning horizon length (i.e., the number of periods). Branch count in these tree structures therefore tends to increase geometrically as a function of node depth (i.e., distance from root node). Thus, total node count tends to increase geometrically as a function of horizon length.

The GCBM model uses 1-year time steps, and simulations may span 100 years or more. Modelling 100 time steps using the current implementation of the `ws3` tree-building algorithm tends to require too much memory to be practical, and the optimization problems become intractable unless time steps

are aggregated (to reduce state tree depth).

The DEG prototype aggregates 1-year time steps into 10-year periods, which allows the model to run much faster and requires less memory, but creates a mis-match between DEG and GCBM timesteps. The SDA algorithm includes functions to disaggregate these 10-year periods to match the 1-year time steps required by GCBM, as well as allocate aspatial disturbance area to specific pixels.

The DEG prototype is a specialized instance of a more general software framework. The prototype includes specific modelling assumptions and parameters that are appropriate for the BC carbon modelling project, but may need to be adjusted if the DEG is adapted for use in a different project.

The harvest schedule optimization problem objective function in the DEG includes two pre-configured modes: *maximize total harvest volume* (`max_hvol`) and *minimize harvest area* (`min_harea`). Additional objective function definitions could be implemented with relative ease, but only these two are available in the current prototype. Constraints in the optimization model include harvest volume targets, harvest area even-flow constraints and periodic harvest volume targets⁹. A single treatment type (clearcut) has been implemented, which resets age to 0 (no change in yield curve after harvesting). All stands are assumed to be operable from age 40 onward. The DEG prototype includes a *utilization rate* parameter associated with harvest treatments, which scales yield curve volume when compiling harvest volume output from the model—the purpose of the utilization rate parameter is to allow simulation of partial removal of merchantable biomass (i.e., some merchantable volume is left on the cutblock, for various reasons). The base `ws3` model is not aware of this utilization rate parameter, so it must be systematically applied to all `ws3` output to correctly estimate biomass volume removal from harvested stands.

One of the key input datasets is periodic harvest volume targets. Harvest volume in the DEG is constrained by prescriptive harvest target values for each MU and period. Harvest volume target values are specified in an input text file (`targets.csv`), which contains other DEG parameter values also indexed by MU and period (e.g., wildfire area target, harvest area even-flow constraint ε , harvest volume target constraint ε , wildfire target area constraint ε).

These volume targets are compiled from long-term harvest projections compiled by provincial government analysts¹⁰ and foresters working for industrial timber licensees. The models used to derive these projections are complex, including a large number of constraints (e.g., riparian buffers, protected areas, minimum old-growth forest and other biodiversity protection constraints, wildlife habitat constraints, etc.). These constraints limit the timing and location of harvesting events simulated in the original projections. We did not have access to any of the original complex models when compiling DEG input

⁹The harvest volume targets get translated to periodic upper and lower bound constraints on harvest volume. The constraints are separated by a relatively tight ε gap whose only purpose is to pro-actively avoid numerical instability issues in the LP solver.

¹⁰BC Ministry of Forests, Lands, Natural Resource Operations & Rural Development, which we will abbreviate as FLNR elsewhere in the text.

datasets, so it is not possible to guarantee that choice of stand type and stand age in the DEG is similar to or coherent with the forest management strategy simulated in the original models—we only have total harvest volume targets, by TSA and period. In an effort to improve the coherence between DEG and FLNR simulations, we implemented a *two-pass unit harvest volume calibration* algorithm, which calibrates the *unit harvest volume* (i.e., the ratio of harvest volume to harvest area, expressed in m^3ha^{-1}) in DEG simulations to more closely match the output of the original FLNR models. See Appendix A2 for details.

2.3.3 Slashburning disturbance modelling details

Slashburning is modelled using the DEG *piggyback* disturbance functions. Each piggyback disturbance specifies a parent disturbance and a proportion parameter. After all parent disturbances have been processed by the SDA algorithm, the DEG allocates piggyback disturbance pixels by selecting a random subset of parent disturbances pixels (at each timestep). The proportion parameter controls the proportion of parent pixels selected. The current DEG prototype sets the slashburning piggyback proportion parameter by MU^{11} .

3 Prototype implementation

This section documents some of the technical details of the software implementation of the current DEG prototype. Note that the DEG is built on top of a flexible Python software library, which makes it relatively easy to re-implement the DEG to suit changing needs.

The current DEG prototype runs on a virtual machine (VM) server in the Compute Canada (CC) Cloud¹², running Ubuntu Linux. We use JupyterHub¹³ software to provide a secure web portal, so multiple users can log into independent interactive DEG sessions simultaneously. We configured the JupyterHub portal to use GitHub authentication functions. This requires DEG prototype users to sign up for a free GitHub account¹⁴. The DEG system administrator then manually configures an account on the DEG server to match the GitHub username, copies appropriate files to the `notebooks` subdirectory in the new user home directory, and adds the username to the global JupyterHub whitelist.

¹¹For the PICS BC project, we are assuming that stands in the *maritime pacific* ecozone have a 0.15 slashburn proportion, and 0.50 in other ecozones. Some MUs boundaries overlap two or more ecozones, so for these cases we compiled an area-weighted average of 0.15 and 0.50 values, based on the proportion of each MU in maritime pacific and other ecozones. We implemented a separate notebook that automates the process of overlaying MU and ecozone boundary data layers, and calculating the area-weighted coefficients.

¹²See <https://www.computecanada.ca/>.

¹³See <https://jupyterhub.readthedocs.io/>.

¹⁴See <https://github.com/> for more information on GitHub, and to sign up for a free account. GitHub username must consist only of lowercase letters [a-z] to be compatible with the current DEG prototype implementation (this constraint may be relaxed in a future release).

It is possible for the system administrator to configure the DEG so that some project directories are shared amongst users (e.g., the directory containing initial forest inventory can be shared so that some DEG users always have matching starting inventory, or multiple users could distribute the task of running different *scenarios* and have all output converge to a shared directory).

Running the DEG on a CC VM like this has a few technical advantages. First, and most importantly, CC cloud computing resources are available to researchers in Canada free of charge—researchers must periodically request, and be granted, access to resources, but they are never billed directly for either access to or utilization of these resources. The current DEG prototype runs on a VM running Ubuntu 16 OS, configured with 16 CPU cores and 120 GB of RAM. The cost of purchasing and operating a similar (hardware) server, or renting a similar VM (e.g., from the Amazon Web Services platform), would be substantial. Another advantage of the CC VM server is that administration and maintenance of the underlying software environment is centralized. This also avoids requiring DEG users to set up and configure the DEG software environment. The prototype currently require installation and configuration of a working Jupyter Notebooks environment, installation of several open-source Python packages, and configuration of a specific file directory structure (to accommodate the complex input and output requirements of the DEG)—we hope to make future releases of the DEG prototype easier to install and configure, but for now the centralized server-based web interface (via JupyterHub) is the simplest way to circumvent these issues.

We use a Jupyter Labs¹⁵ environment to provide an interactive web-based notebook interface to the DEG system. With Jupyter notebooks, we can interleave text (explaining what we are about to do), Python code (defining parameters, functions and operations that collectively implement the DEG), interactive controls (e.g., buttons, sliders, text input boxes, etc.) and output (e.g., console printout, tables of data, figures, error messages, etc.). Jupyter notebooks can easily import functions from system-wide Python software packages installed on the server, call functions implemented locally in the project working directory, or use functions defined in the notebook itself. This makes the notebook environment very flexible, which works well in a rapid prototyping agile development environment.

The current prototype models the entire public forest of BC (38 MUs), using 100×100 m square pixels (i.e., 1 ha pixels) and 1-year time steps for GeoTIFF output interface with GCBM. GeoTIFF output is georeferenced using NAD83/BC Albers Equal Area projected coordinate system¹⁶—this coordinate system preserves area, so that DEG pixels are all the same size regardless of where they are situated in the province of BC. A different coordinate system may be more appropriate if the DEG prototype is adapted for use in a geographic area other than BC.

We model three disturbance types (clearcut harvesting, wildfire, and slash-

¹⁵Via See <http://jupyter.org/>.

¹⁶See <https://epsg.io/3005> for details.

burning). The geographic projection and pixel size must be set in the DEG software to match the projection and resolution used in GCBM.

Many of the core functions of the DEG, including the aspatial forest modelling functions and SDA functions, are implemented in the **ws3** open-source software package. The **ws3** package can be readily expanded to accommodate changing needs, and is used in a number of projects outside the DEG to model landscape-level forest management problems in a variety of contexts (including wood supply analysis, supply chain modelling, forest pest modelling, etc.).

Documentation for the **ws3** package can be found online¹⁷. Source code can be downloaded from GitHub¹⁸. The documentation, which includes a general overview of the package and detailed technical documentation of the application programming interface (API), is compiled from docstrings embedded in the source code, and is automatically updated every time the source on GitHub is updated. Most of the functions used to model the aspatial disturbance scheduling step in the DEG process are implemented in the **ws3.forest** module. Most of the functions used to model the SDA step in the DEG process are implemented in the **ws3.spatial** module.

The **ws3** package is currently only configured to use the Gurobi solver¹⁹), but is designed in anticipation of a future need to interface with other solvers.

4 Acknowledgements

The work documented in this report was supported by funding from the *Pacific Institute for Climate Solutions* (PICS).

References

- Gunn, E. (2007). Models for strategic forest management. In Weintraub, A., Romero, C., Bjørndal, T., and Epstein, R., editors, *Handbook of Operations Research in Natural Resources*, chapter 16, pages 317–341. Springer Science+Business Media.
- Savage, D. W., Martell, D. L., and Wotton, B. M. (2010). Evaluation of two risk mitigation strategies for dealing with fire-related uncertainty in timber supply modelling. *Canadian Journal of Forest Research*, 40(6):1136–1154.

¹⁷See <http://ws3.readthedocs.io/>.

¹⁸See <https://github.com/gparadis/ws3/>.

¹⁹See <http://www.gurobi.com/> for more information on Gurobi. Reasons for selecting the Gurobi solver include performance (it is one of the top two solvers available), elegant Python bindings (makes it relatively easy to integrate with **ws3**, and convenient academic licencing policy (Gurobi solver can be used freely for research and teaching, and license deployment to new installations is relatively simple, although it still requires some moderately complex one-time operations on each new setup to activate the solver license).

Appendices

A1 Optimization model formulation

Harvest scheduling is modelled using a Model I linear programming (LP) optimization problem. See Gunn (2007) for an overview of model formulations commonly used in a strategic forest management planning context.

The optimization problem can be formulated as follows

$$\max \sum_{i \in I} \sum_{j \in J_i} c_{ij} x_{ij} \quad (1)$$

$$\text{s.t.} \quad (2)$$

$$(1 - \varepsilon_p) y_p \leq \sum_{i \in I} \sum_{j \in J_i} \mu_{ijpt} x_{ij} \leq (1 + \varepsilon_p) y_p, \quad \forall p \in O', t \in T'_p \quad (3)$$

$$v_{ot}^- \leq \sum_{i \in Z} \sum_{j \in J_i} \mu_{ijot} x_{ij} \leq v_{ot}^+, \quad \forall o \in O, t \in T \quad (4)$$

$$\sum_{j \in J_i} x_{ij} = 1, \forall i \in Z \quad (5)$$

$$0 \leq x_{ij} \leq 1, \quad \forall i \in Z, j \in J_i \quad (6)$$

where

I := set of spatial zones

J_i := set of available prescriptions for zone $i \in I$

O := set of forest outputs

$O' \subseteq O$:= set of targeted forest outputs

T := set of time periods in the planning horizon

$T'_p \subseteq O$:= subset of T on which even-flow constraints for output $p \in O'$ are applied

ε_p := admissible level of variation on yield of targeted output $p \in O'$

μ_{ijot} := quantity of output $o \in O$ produced in period $t \in T$ by prescription $j \in J_i$ in zone $i \in I$

μ_{ijpt} := quantity of output $p \in O'$ produced in period $t \in T$ by prescription $j \in J_i$ in zone $i \in I$

v_{ot}^- := lower bound on yield of output $o \in O$ in period $t \in T$

v_{ot}^+ := upper bound on yield of output $o \in O$ in period $t \in T$

c_{ij} := objective function contribution of prescription $j \in J_i$ in zone $i \in I$

x_{ij} := proportion of zone $i \in Z$ on which prescription $j \in J_i$ is applied

$y_p = \sum_{i \in I} \sum_{j \in J_i} \mu_{ijpt_p^R} x_{ij}$, i.e. total yield of targeted output $p \in O'$ at reference period t_p^R

The objective function (1) maximizes the sum of $c_{ij}x_{ij}$ products, which represent yield of a user-defined output—the output can be anything, but is typically set to harvest volume. The variables x_{ij} are linear, with domain $\{x_{ij} \in \mathbb{R} | 0 \leq x_{ij} \leq 1\}$ specified in (6). The set $O' \subseteq O$ represents targeted outputs, for which we enforce even-flow constraints. Even-flow constraints are defined in (3), and are expressed in terms of y_p , which represents total yield of targeted output $p \in O'$ in reference period t_p^R . Note that even-flow constraints are defined over time periods T'_p , which is any subset of T (need not be contiguous)— T'_p can be unique for each output $p \in O'$. General constraints (4) set upper and lower bounds on periodic yield of any output $o \in O$. Coverage constraints (5) require prescriptions to cover the entire zone—doing nothing for the entire planning horizon is considered a prescription that could generate some outputs.

Note that the DEG software framework allows the objective function *sense* to be reversed easily (i.e., minimize instead of maximize), although minimizing would not be logical in the case of harvest volume. Minimizing would be logical for other situations, for example if the objective function minimized harvest area in an effort to concentrate a target harvest volume on the smallest possible area (i.e., minimize disturbed area).

A2 Two-pass unit harvest volume calibration

We were able to obtain²⁰ projected harvest area data by TSA and year for a sample of eight TSAs in BC²¹. We compared the ratio of harvest volume to harvest area (i.e., $\text{m}^3/\text{ha}^{-1}$) for FLNR model output to single-pass DEG model output for both *minimize harvest area* and *maximize harvest volume* objective function formulations (see Figure 5). This analysis revealed that the FLNR unit harvest volume data is highly correlated with and bounded by DEG output under both objective function modes. Indeed, the ratio of FLNR unit harvest volume to DEG `min_harea` harvest volume is relatively stable, with a mean value of 1.35 across all 8 sample TSAs.

We extended the DEG harvest schedule simulation algorithm to include an optional two-pass harvest area factor, which can be used to calibrate the unit harvest volume of DEG output. In the first pass, we solve the basic optimization problem (in `min_harea` mode) to determine a base vector of periodic harvest area values $A = \{a_1, a_2, \dots, a_{|T|}\}$, where T is the set of periods in the model horizon. In the second pass, we further constrain the optimization problem, such that harvest area in each period $t \in T$ is bounded by $Ca_t \pm \varepsilon Ca_t$, where C is a user-defined constant calibration coefficient and ε is a small number.

Using trial and error, we iteratively derived a value of $C = 1.50$, which greatly reduces the difference in unit harvest volume between two-pass calibrated DEG simulations and FLNR simulations—see Figure 5 for a comparison of DEG and FLNR unit harvest volumes before and after calibration.

²⁰Qinglin Li, FLNR, personal communication, 2018-08-03.

²¹Out of a total of 38 TSAs.

In summary, we calibrated the DEG to more closely approximate the FLNR unit harvest volume for the eight sample TSAs for which we had harvest area data. We use $C = 1.50$ for all TSAs for the final DEG simulations—note that we have no way of verifying the effectiveness of the calibration outside of the eight sample TSAs, so this approach implicitly assumes that the sample of eight TSAs we used for calibration is representative of the full population. If we had a larger training dataset, we may be able to further refine the calibration coefficient value (or apply different calibration coefficients to clusters of similar TSAs, for example interior versus coastal, etc.).

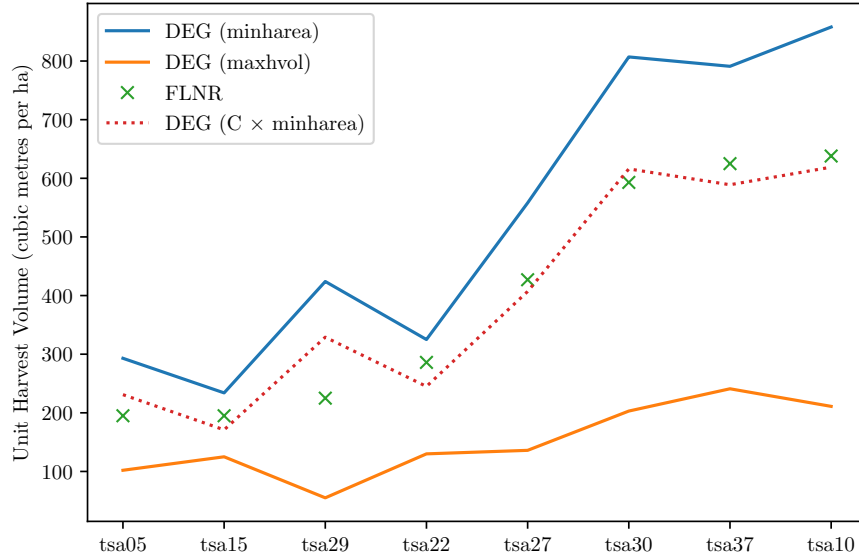


Figure 5: Comparison of calibrated DEG and FLNR model unit harvest volumes, for eight sample TSAs. FLNR model output shown with $\times$ symbols. Solid lines show single-pass DEG output under both *minimize harvest area* and *maximize harvest volume* objective function modes. Dotted line shows calibrated DEG output.

A3 DEG output file format notes

By convention, output for each DEG scenario is coallated in a directory named after the scenario. DEG and GCBM prototypes run on different servers, so we ZIP each scenario directory and copy these files to a CFS FTP server to stage the files for transfer to the GCBM server. The text below contains a detailed description of the ZIP archive contents.

Each DEG output ZIP archive contains 230 files, for one scenario (6 files per TSA, plus two global diagnostic reports). Most of these files contain diagnostic

output from the DEG, which are included to help validate that the linkage with GCBM is working correctly. We provide a full list of filename patterns below, with brief descriptions of contents. **sname** refers to the scenario codename²². **tsa00** refers to one of 38 TSA basenames (see Appendix A3.3 for a list of valid basenames used in the DEG prototype). The term *period* implies 10-year modelling periods.

sname.tsa00.distout.zip

GeoTIFF disturbance data for GCBM. One file per TSA. Each archive contains one GeoTIFF file for each combination of disturbance type and year. File-name pattern is like **projected.dname.0000.tif**, where **dname** is disturbance name (i.e., one of **harvest**, **fire**, **slashburn**) and 0000 is year (in the range [2015, 2074]). Each GeoTIFF files contains one layer, with pixel values 0 (not disturbed) or 1 (disturbed).

sname.tsa00.diag1.csv

Diagnostic data, in tabular format. One file per TSA. One row per period. Columns correspond to period, aspatial harvest area (schedule), aspatial harvest area (simulated), aspatial fire area, aspatial harvest volume, ratio of harvest volume to harvest area (i.e., unit harvest volume).

sname.tsa00.diag1.png

Diagnostic data, in image format. One file per TSA. Figure comparing the two harvest area columns from the matching **diag1** tabular report (i.e., scheduled versus simulated harvest area). These should be nearly identical in most cases.

sname.tsa00.diag2.png

Diagnostic data, in image format. One file per TSA. Three sub-figures. The first two sub-figures plot aspatial periodic disturbance target area next to year-wise sum of disturbance pixel areas (i.e., comparison of aspatial modelling step with output of final SDA step in the DEG algorithm), for harvest and fire disturbances. These should be coherent. Minor deviations are expected. The third subfigure shows relative target shortfall for both harvest and fire disturbances.

sname.tsa00_fm.pkl

Serialized snapshot of **ws3.forest.ForestModel** instance created by the DEG. Can be unpickled in a subsequent Python session and used to run diagnostic queries, or for further simulation.

²²Scenario codenames currently implemented in the DEG include **base**, **rogh**, and **more**. See Appendix A3.2 for details.

`sname_tsa00_sch.csv`

Serialized snapshot of final aspatial combined harvest and fire schedule generated by the DEG. Can be used to re-create the state the state that the `ws3.forest.ForestModel` instance was in just before running the SDA step (or for compiling diagnostic reports).

`sname_distout_area.csv`

Summary report listing disturbance pixel area by scenario, TSA, disturbance type, and year (ha).

`sname_distout_vcut.csv`

Summary report listing harvest volume by scenario, TSA, and period (m³). Volume values are scaled by scenario-wise utilization coefficient.

A3.1 DEG input data pre-processing functions

The DEG prototype features a number of input data pre-processing functions. Specifically, the DEG has functions for pre-processing the spatial stand inventory data. For each MU, we must provide a spatial data layer describing the geometry and attributes of each stand, at year 0 of the simulation.

The DEG prototype requires this stand inventory input to be in a specific rasterized GeoTIFF data format. The DEG uses the cell size, cell row and column counts, and coordinate reference system parameters from this input data layer to automatically set parameters for the output GeoTIFF disturbance files. The original stand inventory data we obtained for BC is encoded in vector-based ESRI geodatabase (GeoDB) format²³.

The original GeoDB datasets for each MU have certain quality control problems (e.g., invalid polygon geometry topology, inconsistent field names, etc.), missing certain attribute fields (e.g., MU codename), and are in the wrong projection. So, we implemented a function in the DEG that automates the process of cleaning up the original vector stand inventory datasets to acceptable quality standards. This function is somewhat CPU-intensive, so it is run once for each MU and the cleaned vector datasets are cached to disk for re-use at the start of each simulation. The vector data cleaning also includes experimental geometry compression features (which reduce the number of vertices required to store each stand polygon while preserving polygon topology relative to neighbouring stands), however testing has revealed that this feature is somewhat unstable and it is disabled by default in the production prototype.

At the start of a simulation for a given MU, the DEG loads the cleaned vector inventory dataset into memory and converts this to the required GeoTIFF rasterized format (using the pixel size and CRS parameters specified in

²³See <http://www.loc.gov/preservation/digital/formats/fdd/fdd000294.shtml> for more information on the GeoDB file format

DEG input parameter files). The rasterized inventory files are cached to disk in GeoTIFF format—technically, it is possible to run the inventory rasterization function only once for each MU and re-use the cached files on disk, but rasterization is relatively fast so we have the production prototype configured to re-run the the inventory rasterization function by default at the start of each simulation.

A3.2 DEG scenario definitions

The DEG prototype developed for BC includes three pre-defined scenarios. Running the DEG with parameter `scenario_name` set to one of `'base'`, `'more'`, or `'rogh'` will induce a cascading update of all other parameters required to model different scenarios.

`base`

The `base` scenario name triggers simulation of the *base* scenario. Utilization rate is set to 0.85.

`more`

The `more` scenario name triggers simulation of the *higher utilization* scenario. Utilization rate is set to 0.90.

`rogh`

The `rogh` scenario name triggers simulation of the *restricted old-growth harvesting* scenario. Utilization rate is set to 0.85 (same as `base` scenario).

The DEG includes functions for automating the process of compiling an *old-growth index* (OGI) curve for each development type in the model. These OGI curves are derived from the total volume yield curves, and express the relative *old-growthedness* of each development type as a function of stand age, in the range $[0, f_{\max}]$. Each OGI curve is flat at extreme left and right, and linearly ramps up between 0 and a user-defined maximum value $f_{\max}$ over an interval bounded by ages $a_1 = C_{1a}a_{\text{MAI}_{\max}} + C_{1b}$ and $a_2 = C_{2a}a_{\text{V}_{\max}} + C_{2b}$, where $a_{\text{MAI}_{\max}}$ is the age at which the *mean annual increment*²⁴ (MAI) of the total volume curve culminates²⁵, $a_{\text{V}_{\max}}$ is the age at which the total volume curve culminates²⁶, and default coefficient values²⁷ are $C_{1a} = 1$, $C_{1b} = 0$, $C_{2a} = 1$, and $C_{2b} = 0$.

²⁴MAI of a yield function $f(x)$ is defined as $g(x, f(x)) = f(x)/x$.

²⁵Automatically calculated from the total volume yield curve by `ws3`.

²⁶Automatically calculated from the total volume yield curve by `ws3`.

²⁷Can be overridden by the user.

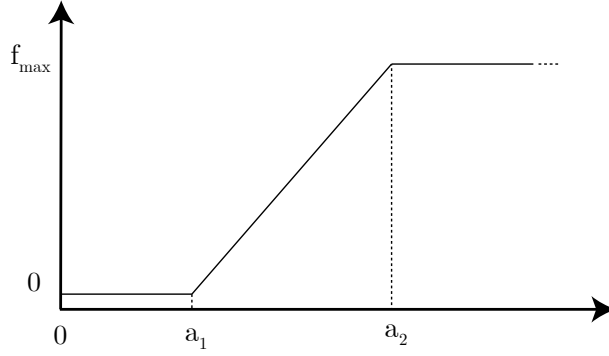


Figure 6: Illustration of DEG OGI curve shape, with inflection points defined by ages a_1 and a_2 .

For the **rough** scenario, harvesting is limited to stands whose OGI curve value is less than 1. A side-effect of this is that all stands older than a_2 at the start of the simulation can never be harvested under this scenario.

A3.3 DEG basename values

The DEG prototype developed for BC is divided into 38 TSA-wise subproblems. Table A1 list basename values used in the DEG prototype.

Table A1: TSA basenames and common names for the BC DEG prototype

TSA Code	TSA Name
tsa01	Arrow
tsa02	Boundary
tsa03	Bulkley
tsa04	Cassiar
tsa05	Cranbrook
tsa07	Golden
tsa08	Fort Nelson
tsa09	Invermere
tsa10	Kalum
tsa11	Kamloops
tsa12	Kispiox
tsa13	Kootenay Lake
tsa14	Lakes
tsa15	Lillooet
tsa16	MacKenzie
tsa17	Robson Valley
tsa18	Merritt
tsa19	Mid Coast
tsa20	Morice
tsa21	North Coast
tsa22	Okanagan
tsa23	100 Mile House
tsa24	Prince George
tsa25	Queen Charlotte
tsa26	Quesnel
tsa27	Revelstoke
tsa29	Williams Lake
tsa30	Fraser
tsa31	Soo
tsa33	Kingscome
tsa37	Strathcona
tsa38	Arrowsmith
tsa39	Sunshine Coast
tsa40	Fort St. John
tsa41	Dawson Creek
tsa43	Nass
tsa44	Pacific
tsa45	Cascadia